ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all

"Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with

the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 ... or Q1, Q2 ...), containing the

C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example

g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the

steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will

be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment

submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited. Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1. Objects in the Metro Fare System:

1. SmartCard: Stores card number and balance. It provides functions to check the balance, deduct fares and recharge the card.
2. Passenger: Stores passenger details and category. It identifies whether the passenger is a student, senior citizen or normal passenger.
3. Journey: Stores source zone, destination zone, travel time and fare.
4. Zone: Represents Inner, Middle and Outer zones. Each zone has a different base fare.
5. FareTable: Stores fare values and calculates the fare based on the travelled zone and peak hours.

2. Procedural and Object-Oriented Approaches:

In procedural programming, separate variables and functions are used to calculate fares and update balances. In object-oriented programming, data and functions are grouped into classes. OOP provides better security, reusability and maintenance.

3. Basic OOP Principles:

Principle	Application in the Metro System
Encapsulation	SmartCard keeps balance private and changes it only through validated methods.
Abstraction	A fare calculation method hides the internal calculation steps.
Inheritance	Student and SeniorCitizen can inherit common details from Passenger.
Polymorphism	Different passenger categories can implement different concession calculations.

4. Rules for a Valid Journey:

- The fare must depend on the farthest zone travelled.
- Peak surcharge applies between 8–10 AM and 5–8 PM.
- The balance after deduction must be at least Rs. 50.
- The card balance must never become negative.
- The fare must be calculated accurately.

Alternative Concept and Complexity Analysis

In procedural programming, global variables store the balance and fare, while separate functions calculate and deduct fares. Both procedural and OOP approaches take O(1) time per journey and O(1) memory per card. However, OOP requires fewer changes when fare rules change because the logic is maintained inside separate classes.

TASK 2 – USE TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS CORRECTLY

1. Five Types of Tokens:

Token Type	Example	Purpose
Keywords	int, class, if, return	Reserved words defining program structure
Identifiers	balance, fare, SmartCard	Names of variables, functions and classes
Constants	1000, 2000, 5000	Fixed values
Operators	+ , - , * , / , ==	Perform calculations and comparisons
Punctuators	; , { } , ( )	Separate and group program elements

2. Types of Constants:

- #define: Creates a macro without a data type.
- const: Defines a value that cannot be modified.

- constexpr: Defines a compile-time constant.

constexpr is preferred because it provides type checking and allows fare values to be calculated at compile time.

3. Enum and Enum Class

```
enum class Zone {
    Inner,
    Middle,
    Outer
};
```

enum class is safer because it provides scope and prevents accidental conversion to integers.

4. Identifier Naming Rules

Identifiers must begin with a letter or underscore. They cannot contain spaces or use reserved keywords. Meaningful names such as smartCardBalance and baseFare improve readability.

Alternative Concept and Complexity Analysis

Using #define instead of constexpr removes type checking and scope protection. Macros occupy no memory, while constexpr values usually require no runtime storage. Both approaches have O(1) execution time, but constexpr is safer and easier to maintain.

TASK 3 – CHOOSE DATA TYPES AND CHECK TYPE COMPATIBILITY

1. Suitable Data Types:

Value	Recommended type	Reason
Fare and balance	long long	Stores integer paise and supports large totals
Hour / time	int	Stores hour values such as 0–23
Zone	enum class	Represents a fixed set of zones safely
Journey count	long long	Supports large daily totals
Peak status	bool	Stores true or false
Passenger name	string	Stores text

2. Why Integer Paise Is Used

Money is stored in integer paise instead of double rupees because floating-point values may produce rounding errors. For example, Rs. 10.50 is stored as 1050 paise. This ensures accurate fare calculations.

3. Type Compatibility:

C++ automatically converts compatible data types during expressions. For example, an integer can be converted into a double when used with a double value. A bool value can also be converted into 0 or 1.

4. Integer Range:

The <climits> header provides limits such as INT_MAX and LLONG_MAX. A 32-bit int can store up to 2,147,483,647, while long long supports much larger values.

For 3,000,000 journeys with a maximum fare of Rs. 36, the total is Rs. 10,80,00,000. Therefore, int is sufficient for this particular total in paise only if the total is 1,080,000,000 paise; however, long long is safer for future expansion.

Alternative Concept and Complexity Analysis

Using double rupees provides decimal values but may introduce rounding errors. Both double and long long generally use 8 bytes and require O(1) time per addition. Integer paise is more reliable because it avoids floating-point precision errors.

TASK 4 – CONTROL IMPLICIT CONVERSIONS

1. Implicit Conversions:

Implicit conversion happens automatically when C++ converts one data type into another.

Examples:

- Integer division removes the decimal part.
- An integer can be converted into a double.
- A character can be converted into an integer.
- An integer can be converted into a Boolean value.
- Signed integers may be converted into unsigned integers during comparison.

2. Integer Division and Double Conversion:

```
int a = 7, b = 2;
double c = a / b;
double d = static_cast<double>(a) / b;
```

Here, c becomes 3.0 because integer division happens first. d becomes 3.5 because a is converted into double before division.

3. Narrowing Conversion:

```
int x{3.7};
```

This produces a compilation error because brace initialization prevents narrowing conversions.

4. static_cast:

static_cast is used to convert a value explicitly from one data type to another.

Predicted Output

```
3  3.5
C  68
1  true
s >= u
```

Explanation

1. Integer division produces 3, while explicit conversion produces 3.5.
2. Character A + 2 produces C. The expression ch + 1 promotes the character to an integer and produces 68.
3. Integer 5 converts to Boolean true, printed as 1 and true.
4. The negative signed integer is converted to unsigned during comparison, so the condition s < u is false.

Alternative Concept and Complexity Analysis

Using static_cast makes conversions explicit and improves code safety. Brace initialization prevents accidental narrowing. Both methods have O(1) time complexity and generally use the same memory. Some conversion errors can be caught during compilation.

TASK 5 – WRITE CORRECT EXPRESSIONS WITH THE RIGHT PRECEDENCE

1. Peak Hour Expression:

```
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)
```

This checks whether the journey occurs during morning or evening peak hours.

2. Surcharge Calculation:

```
fare = fare * 120 / 100;
```

This increases the fare by 20%.

3. Concession Calculation:

The concession percentage is not specified in the scenario. Therefore, it should be stored as a separate configurable constant.

```
fare = fare * (100 - concessionPercent) / 100;
```

4. Increment Operators:

Operator concept	Meaning	Example
x++	Use old value, then increment	int z=x++
++x	Increment, then use new value	int z=++x
Multiplication precedence	Multiplication before addition	x + y * 2
Parentheses	Evaluate grouped expression first	(x+y) * 2
Remainder	Returns signed remainder	-10 % 3 gives -1

5. Division and Modulus:

Integer division removes the decimal part. The modulus operator returns the remainder. For negative numbers, the remainder follows the sign of the dividend.

Predicted Output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Explanation

1. x + y * 2 gives 11 because multiplication has higher precedence. (x + y) * 2 gives 16 because parentheses are evaluated first.
2. x++ + ++y gives 9. Then x becomes 6 and y becomes 4.
3. The conditional operator returns 6. The modulus and multiplication expression gives 4.
4. Integer division and modulus produce 3, 1, -3 and -1.
5. The loop adds 0 + 3 + 6 + 9, giving 18.

Alternative Concept and Complexity Analysis

A Boolean lookup table can store peak-hour information for all 24 hours. It requires 24 bytes of memory if each Boolean occupies one byte. Both the lookup table and logical expression have O(1) time complexity. The lookup table reduces repeated comparisons.

TASK 6 – DESIGN THE CONTROL STRUCTURES

1. Control Structures:

- if-else: Checks peak hours and card balance.
- switch: Selects the base fare according to the zone.
- Conditional operator: Selects between two values.
- for loop: Processes a fixed number of journeys.
- range-based for loop: Processes every journey in an array.
- while loop: Continues processing until the input ends.
- break: Exits a switch or loop.
- continue: Skips an invalid journey.

2. Pseudocode:

```
START
INPUT card balance
INPUT journey details
FOR each journey
    CHECK whether zone is valid
    IF zone is invalid
        SKIP journey
    ENDIF
    SELECT base fare using zone
    CHECK peak hour
    IF peak hour
        ADD 20% surcharge
    ENDIF
    IF balance - fare >= 5000
        DEDUCT fare
        DISPLAY fare and balance
```

```
    ELSE
        DISPLAY journey denied
    ENDIF
END FOR
STOP
```

3. Complete the Code:

```
#include <iomanip>
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
    int base = 0;
    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
        default:
            return -1;
    }
    if (peak)
```

```
        base = base * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    return base;
}
bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
int main() {
   long long balance = 10000;
    struct Trip {
        Zone zone;
        bool peak;
    };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            cout << "Fare Rs " << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');
        } else {
            cout << "Denied (low balance)";
        }
        cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Expected Output

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Peak Boundary Testing

Time	Peak status
8:00 AM	Peak
10:00 AM	Off-peak
5:00 PM	Peak
8:00 PM	Off-peak

Alternative Concept and Complexity Analysis

An array can replace the switch statement to retrieve fares using zone indexes. Both methods take O(1) time. The array requires O(1) additional space for three fare values. A fourth zone can be added by extending the array or adding another switch case.

TASK 7 – FROM PROCEDURAL TO OBJECT-ORIENTED

1. Procedural Programming:

```
#include <iostream>
using namespace std;
long long balance = 10000;
const long long MIN_BALANCE = 5000;
bool canTravel(long long fare) {
    return balance - fare >= MIN_BALANCE;
}
void deduct(long long fare) {
```

```
    if (canTravel(fare))
        balance -= fare;
    else
        cout << "Journey Denied\n";
}
int main() {
    deduct(1000);
    cout << "Balance: " << balance / 100.0;
}
```

2. Object-Oriented Programming:

```
#include <iostream>
using namespace std;
class SmartCard {
private:
    long long balance;
    const long long MIN_BALANCE = 5000;
public:
    SmartCard(long long b) {
        balance = b;
    }
    bool canTravel(long long fare) {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }
    void deduct(long long fare) {
        if (canTravel(fare))
            balance -= fare;
        else
            cout << "Journey Denied\n";
```

```
    }
    long long getBalance() {
        return balance;
    }
};
int main() {
    SmartCard card(10000);
    card.deduct(1000);
    cout << "Balance: " << card.getBalance() / 100.0;
}
```

3. Comparison:

Procedural programming uses separate variables and functions. OOP combines data and functions inside a class. The private balance prevents direct modification and protects the minimum balance rule.

OOP is easier to maintain because changes can be made inside the SmartCard class without affecting the entire program.

Alternative Concept and Complexity Analysis

If balance were public, any part of the program could change it directly and create a negative balance. Both designs have O(1) time and O(1) space complexity. Private data members provide better security through encapsulation.

TASK 8 – READ AND PRINT DATA SAFELY

1. Input Handling:

The cin statement reads user input. The cin.fail() function checks whether the input operation failed.

```
int hour;
cin >> hour;
if (cin.fail()) {
    cin.clear();
    cin.ignore(1000, '\n');
    cout << "Invalid input";
}
```

2. Output Formatting:

The <iomanip> header provides formatting functions.

- setw(): Sets the output width.
- setfill(): Fills empty spaces.
- fixed: Displays decimal values.
- setprecision(): Controls decimal places.

3. Example Program:

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double fare = 36.0;
    cout << fixed << setprecision(2);
    cout << "Metro Fare: Rs " << fare;
    return 0;
}
```

Output:

```
Metro Fare: Rs 36.00
```

4. Journey Report:

A journey report should contain the card number, journey zone, travel time, fare deducted and remaining balance. This helps passengers and metro administrators verify transactions.

Alternative Concept and Complexity Analysis:

Using getline() reads an entire input line, while stoi() converts it into an integer. Try-catch handles invalid input. Both methods require O(L) time for a line of length L. getline() needs additional buffer space, while cin directly reads formatted values.

TASK 9 – IMPLEMENT AND TEST THE FARE LOGIC

1. Test Cases:

Test	Zone	Time	Expected fare
1	Inner	11 AM	Rs. 10
2	Middle	9 AM	Rs. 24
3	Outer	6 PM	Rs. 36
4	Inner	8 AM	Rs. 12
5	Outer	8 PM	Rs. 30
6	Middle	10 AM	Rs. 20
7	Middle	5 PM	Rs. 24
8	Outer	12 PM	Rs. 30

2. Boundary Conditions:

- At 8 AM, the 20% surcharge applies.
- At 10 AM, the surcharge does not apply.
- At 5 PM, the surcharge applies.
- At 8 PM, the surcharge does not apply.
- A journey is accepted if exactly Rs. 50 remains after deduction.
- Negative recharge values must be rejected.
- Invalid zones and non-numeric hours must be rejected.

3. Five-Journey Calculation:

Assume five journeys:

1. Inner off-peak = Rs. 10
2. Middle off-peak = Rs. 20
3. Outer off-peak = Rs. 30
4. Inner peak = Rs. 12
5. Outer peak = Rs. 36

Total fare:

Rs. 10 + Rs. 20 + Rs. 30 + Rs. 12 + Rs. 36 = Rs. 108.

Alternative Concept and Complexity Analysis

A test-case table can store inputs and expected outputs. A loop checks every test case and compares actual results with expected results. For t test cases, time complexity is O(t), and space complexity is O(t). New boundary cases can be added easily.

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION

1. Journey Processing Growth:

The metro processes different numbers of journeys:

- One station: 20,000 journeys per day.
- One line: 500,000 journeys per day.
- Whole network: 3,000,000 journeys per day.

2. Integer and Double Comparison:

Integer paise provides exact monetary calculations, while double may introduce small rounding errors.

For example, adding Rs. 0.10 repeatedly using floating-point arithmetic may produce an approximate result instead of an exact decimal total.

3. Maximum Fare Calculation:

Assuming every journey costs Rs. 36:

Network level	Journeys/day	Total rupees	Total paise
One station	20,000	Rs. 7,20,000	72,000,000
One line	500,000	Rs. 1,80,00,000	1,800,000,000
Whole network	3,000,000	Rs. 10,80,00,000	10,800,000,000

The 3,000,000-journey total is 10,800,000,000 paise, which exceeds the maximum signed 32-bit int value of 2,147,483,647. Therefore, int is not safe for the network total. long long is recommended.

Integer-paise totals are exact. Double totals may show small floating-point differences, depending on the values and accumulation order. Compiler options -Wall -Wextra can report issues such as unused variables, signed/unsigned comparisons and suspicious conversions. Actual warnings depend on the code and compiler.

Type	At 3 million journeys	Assessment
int	Can overflow when storing paise total	Not suitable
long long	Stores 10,800,000,000 paise safely	Recommended
double	Large enough, but decimal precision is approximate	Avoid for exact money

Alternative Concept and Complexity Analysis

- int: May overflow when storing the complete network total in paise.
- long long: Safely stores the total.
- double: Stores large values but may introduce precision errors.

All three types generally require O(1) time per addition. Long long is recommended for accurate money calculations.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

1. Recommended Data Types:

- long long for fare and balance in paise.
- int for time and zone indexes.
- bool for peak-hour status.
- enum class for metro zones.
- constexpr for fare values and minimum balance.

2. Recommended Classes:

The final system should contain SmartCard, Passenger, Journey and FareTable classes. These classes separate responsibilities and make the program easier to maintain.

3. Fare Revision:

If the metro changes its fare, only the fare constants or fare table need to be updated. The SmartCard class can continue using the same deduction logic.

4. Limitations:

The original design supports only three zones and a fixed peak surcharge. Adding more zones, discounts and monthly passes requires modifications.

5. Whole Network Support:

The system can support 3 million journeys per day using efficient O(1) fare calculation and long long for monetary totals. Each journey can be processed independently.

Alternative Concept and Complexity Analysis

A configuration file can store fare values and load them when the program starts. Reading z fare values requires O(z) time and O(z) space. This avoids recompiling the program whenever fares change, but the file must be validated to prevent incorrect fare values.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

A. Impact Analysis:

The metro introduces three new requirements:

1. A fourth zone called Airport with a base fare of Rs. 60.
2. A 10% weekend discount that cannot be combined with the 20% peak surcharge.
3. A monthly pass that allows unlimited journeys in two selected zones.

The following components must be modified:

- Zone enum must include Airport.
- Fare table must include Rs. 60.

- Peak and weekend fare calculations must be updated.
- SmartCard must support monthly pass validation.
- A new Pass class must store the two selected zones.

B. Evaluation of the Original Design:

The original switch-based fare logic can support Airport by adding another case. However, adding more zones makes the switch statement longer.

The SmartCard class can still manage balances, but it needs additional logic to check monthly passes.

Worked examples:

Journey	Calculation	Final fare
Inner weekday peak	Rs. 10 + 20%	Rs. 12
Airport weekday peak	Rs. 60 + 20%	Rs. 72
Airport weekend	Rs. 60 - 10%	Rs. 54
Inner weekend	Rs. 10 - 10%	Rs. 9
Pass-covered zone	Monthly pass covers zone	Rs. 0

The weekend discount is applied instead of the peak surcharge, never both.

C. Adaptation and Redesigned C++ Code:

The revised system uses a fare table, separate weekend discount logic and a Pass class.

```
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long FARE[] = {
    1000, 2000, 3000, 6000
};
class Pass {
private:
    Zone zone1, zone2;
public:
```

```
    Pass(Zone a, Zone b) {
        zone1 = a;
        zone2 = b;
    }
    bool covers(Zone z) {
        return z == zone1 || z == zone2;
    }
};
class SmartCard {
private:
    long long balance;
    const long long MIN_BALANCE = 5000;
public:
    SmartCard(long long b) {
        balance = b;
    }
    bool canTravel(long long fare) {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }
    void deduct(long long fare) {
        if (canTravel(fare))
            balance -= fare;
        else
            cout << "Journey Denied\n";
    }
    long long getBalance() {
        return balance;
    }
```

```
};
long long calculateFare(
    Zone z, bool peak, bool weekend, Pass &p) {
    if (p.covers(z))
        return 0;
    int index = static_cast<int>(z);
    long long fare = FARE[index];
    if (weekend)
        fare = fare * 90 / 100;
    else if (peak)
        fare = fare * 120 / 100;
    return fare;
}
int main() {
    SmartCard card(20000);
    Pass monthlyPass(Zone::Inner, Zone::Middle);
    Zone journeys[] = {
        Zone::Inner,
        Zone::Outer,
        Zone::Airport
    };
    for (Zone z : journeys) {
        long long f = calculateFare(
            z, true, false, monthlyPass);
        if (card.canTravel(f)) {
            card.deduct(f);
            cout << "Fare: Rs "
                 << f / 100.0 << '\n';
        } else {
```

```
            cout << "Journey Denied\n";
        }
    }
    cout << "Remaining Balance: Rs "
         << card.getBalance() / 100.0;
}
```

Expected Output

```
Fare: Rs 0
Fare: Rs 36
Fare: Rs 72
Remaining Balance: Rs 92
```

D. Evidence-Based Defence:

The redesigned system is better because:

1. The fare table makes adding new zones easier.
2. Weekend discounts and peak surcharges are calculated separately.
3. The Pass class supports unlimited journeys in two selected zones.
4. SmartCard protects the balance and minimum balance condition.
5. Integer paise prevents floating-point rounding errors.

The revised system provides better flexibility, accuracy and maintainability. It can support future fare changes and additional metro zones with fewer code modifications.

Alternative Concept and Complexity Analysis

Instead of storing fare values directly in the program, they can be loaded from a configuration file. If there are z zones, loading the fare table requires O(z) time and O(z) memory.

This approach allows administrators to update fares without recompiling the program. However, the configuration file must be validated to prevent incorrect fare values.

FINAL CONCLUSION:

The Namma Metro Smart-Card Fare System can be implemented effectively using object-oriented programming principles in C++. Integer paise ensures accurate fare calculations, while classes provide data security and maintainability. The redesigned system supports additional zones, weekend discounts and monthly passes. Therefore, the modified object-oriented design is suitable for future metro network expansion.